Doctrine Deprecations

A small (side-effect free by default) layer on top of `trigger_error(E_USER_DEPRECATED)` or PSR-3 logging.

- no side-effects by default, making it a perfect fit for libraries that don't know how the error handler works they operate under
- options to avoid having to rely on error handlers global state by using PSR-3 logging
- deduplicate deprecation messages to avoid excessive triggering and reduce overhead

We recommend to collect Deprecations using a PSR logger instead of relying on the global error handler.

Usage from consumer perspective:

Enable Doctrine deprecations to be sent to a PSR3 logger:

```
\Doctrine\Deprecations\Deprecation::enableWithPsrLogger($logger);
```

Enable Doctrine deprecations to be sent as `@trigger_error($message, E_USER_DEPRECATED)` messages by setting the `DOCTRINE_DEPRECATIONS` environment variable to `trigger`. Alternatively, call:

```
\Doctrine\Deprecations\Deprecation::enableWithTriggerError();
```

If you only want to enable deprecation tracking, without logging or calling `trigger_error` then set the `DOCTRINE_DEPRECATIONS` environment variable to `track`. Alternatively, call:

```
\Doctrine\Deprecations\Deprecation::enableTrackingDeprecations();
```

Tracking is enabled with all three modes and provides access to all triggered deprecations and their individual count:

```
$deprecations = \Doctrine\Deprecations\Deprecation::getTriggeredDeprecations();

foreach ($deprecations as $identifier => $count) {
    echo $identifier . " was triggered " . $count . " times\n";
}
```

Suppressing Specific Deprecations

Disable triggering about specific deprecations:

```
\Doctrine\Deprecations\Deprecation::ignoreDeprecations("https://link/to/deprecations-descrip
```

Disable all deprecations from a package

```
\Doctrine\Deprecations\Deprecation::ignorePackage("doctrine/orm");
```

Other Operations

When used within PHPUnit or other tools that could collect multiple instances of the same deprecations the deduplication can be disabled:

```
\Doctrine\Deprecations\Deprecation::withoutDeduplication();
```

Disable deprecation tracking again:

```
\Doctrine\Deprecations\Deprecation::disable();
```

Usage from a library/producer perspective:

When you want to unconditionally trigger a deprecation even when called from the library itself then the `trigger` method is the way to go:

```
\Doctrine\Deprecations\Deprecation::trigger(
    "doctrine/orm",
    "https://link/to/deprecations-description",
    "message"
);
```

If variable arguments are provided at the end, they are used with `sprintf` on the message.

```
\Doctrine\Deprecations\Deprecation::trigger(
    "doctrine/orm",
    "https://github.com/doctrine/orm/issue/1234",
    "message %s %d",
    "foo",
    1234
);
```

When you want to trigger a deprecation only when it is called by a function outside of the current package, but not trigger when the package itself is the cause, then use:

```
\Doctrine\Deprecations\Deprecation::triggerIfCalledFromOutside(
    "doctrine/orm",
    "https://link/to/deprecations-description",
    "message"
);
```

Based on the issue link each deprecation message is only triggered once per request.

A limited stacktrace is included in the deprecation message to find the offending location.

Note: A producer/library should never call `Deprecation::enableWith` methods and leave the decision how to handle deprecations to application and frameworks.

Usage in PHPUnit tests

There is a `VerifyDeprecations` trait that you can use to make assertions on the occurrence of deprecations within a test.

```
use Doctrine\Deprecations\PHPUnit\VerifyDeprecations;

class MyTest extends TestCase
{
    use VerifyDeprecations;

    public function testSomethingDeprecation()
    {
        $this->expectDeprecationWithIdentifier('https://github.com/doctrine/orm/issue/1234')

        triggerTheCodeWithDeprecation();
    }

    public function testSomethingDeprecationFixed()
    {
        $this->expectNoDeprecationWithIdentifier('https://github.com/doctrine/orm/issue/1234

        triggerTheCodeWithoutDeprecation();
    }
}
```

Displaying deprecations after running a PHPUnit test suite

It is possible to integrate this library with PHPUnit to display all deprecations triggered during the test suite execution.

```
<phpunit xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:noNamespaceSchemaLocation="vendor/phpunit/phpunit/phpunit.xsd"
         colors="true"
         bootstrap="vendor/autoload.php"
         displayDetailsOnTestsThatTriggerDeprecations="true"
         failOnDeprecation="true"
    >
    <!-- one attribute to display the deprecations, the other to fail the test suite -->

    <php>
        <!-- ensures native PHP deprecations are used -->
        <server name="DOCTRINE_DEPRECATIONS" value="trigger"/>
    </php>

    <!-- ensures the @ operator in @trigger_error is ignored -->
    <source ignoreSuppressionOfDeprecations="true">
```

```
        <include>
            <directory>src</directory>
        </include>
    </source>
</phpunit>
```

Note that you can still trigger Deprecations in your code, provided you use the `#[WithoutErrorHandler]` attribute to disable PHPUnit's error handler for tests that call it. Be wary that this will disable all error handling, meaning it will mask any warnings or errors that would otherwise be caught by PHPUnit.

At the moment, it is not possible to disable deduplication with an environment variable, but you can use a bootstrap file to achieve that:

```
// tests/bootstrap.php
<?php

declare(strict_types=1);

require dirname(__DIR__) . '/vendor/autoload.php';

use Doctrine\Deprecations\Deprecation;

Deprecation::withoutDeduplication();
```

Then, reference that file in your PHPUnit configuration:

```
<phpunit ...
        bootstrap="tests/bootstrap.php"
        ...
    >
    ...
</phpunit>
```

What is a deprecation identifier?

An identifier for deprecations is just a link to any resource, most often a Github Issue or Pull Request explaining the deprecation and potentially its alternative.